

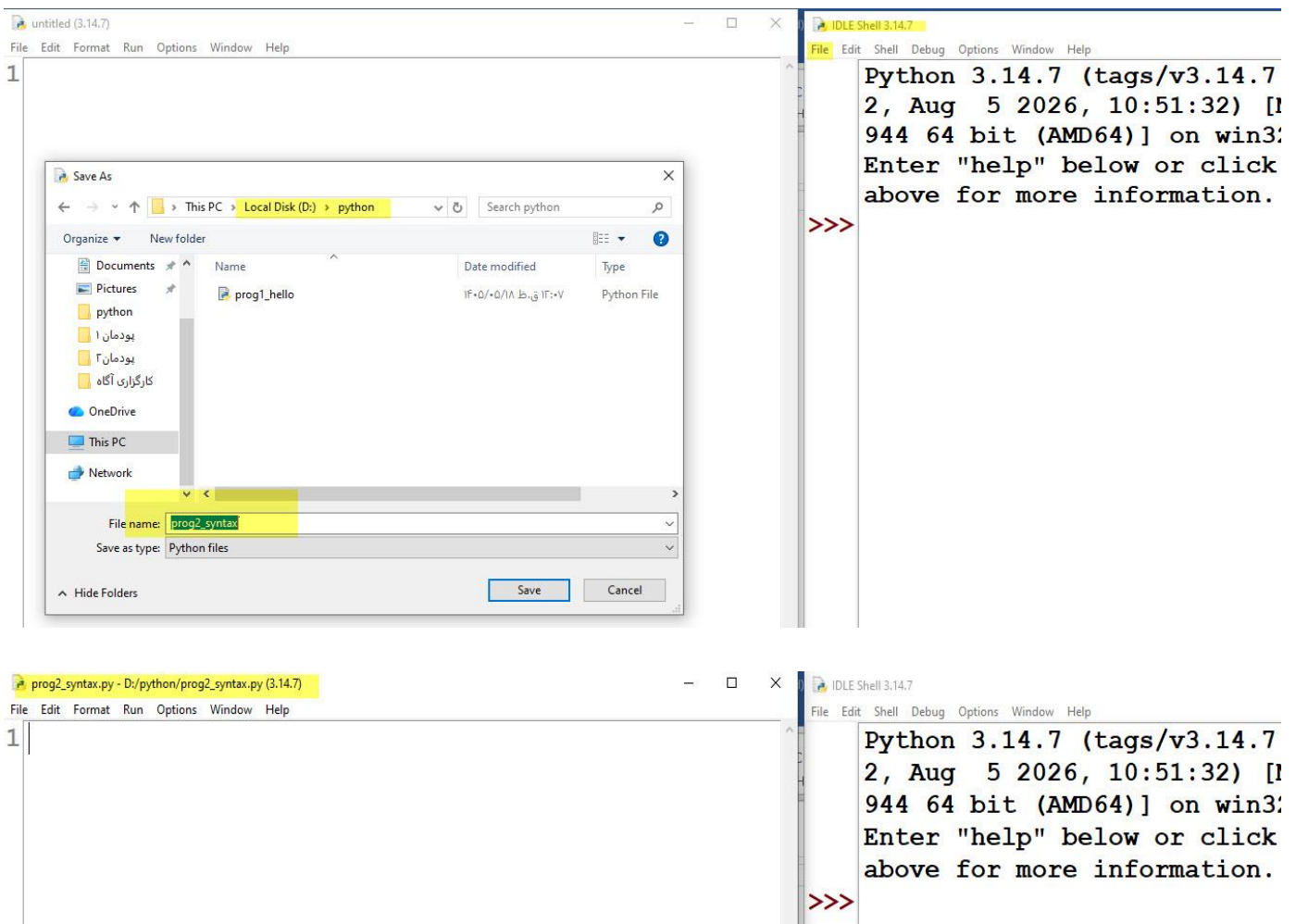
بسمه تعالی

ساختار دستورات در پایتون - رشته ها - تورفتگی ها - چاپ اعداد

در این جلسه آموزشی قصد داریم موضوعات زیر را مورد بررسی قرار دهیم:

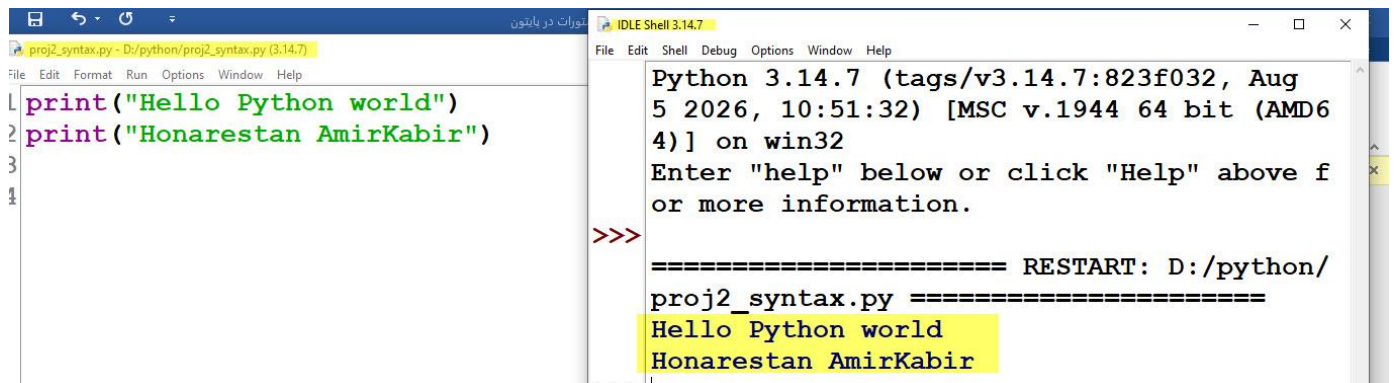
۱- ساختار دستور (Syntax) ۲- تورفتگی (indent) ۳- رشته (string) ۴- اعداد (Numbers)

ساختار دستور (Syntax): ابتدا براساس آنچه تاکنون یاد گرفته ایم با کلیک روی آیکن پایتون (IDLE Python) در نوار وظیفه، پنجره Shell را باز نموده و یک فایل پایتونی جدید در مسیر دلخواه (مثلا فایلی با نام prog2_syntax در پوشه python در درایو D) ایجاد می کنیم. (شکل ۱)



(شکل ۱)

ساختار دستورات زبان پایتون مشابه زبان انگلیسی می باشد اگر به عنوان مثال در پنجره کد نویسی (شکل ۱)، دستور `print("Hello Python world")` را بنویسیم، مشاهده می کنیم که کلمه کلیدی `Print`، که یکی از دستورات اصلی زبان پایتون می باشد بسیار مشابه با ساختار کلمات زبان انگلیسی می باشد و به همین علت است که یادگیری زبان پایتون را برای کاربران راحت می کند زیرا بسیاری از دستورات اصلی زبان پایتون مشابه با ساختار کلمات زبان انگلیسی می باشد. (شکل ۲)



```

Python 3.14.7 (tags/v3.14.7:823f032, Aug 5 2026, 10:51:32) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.

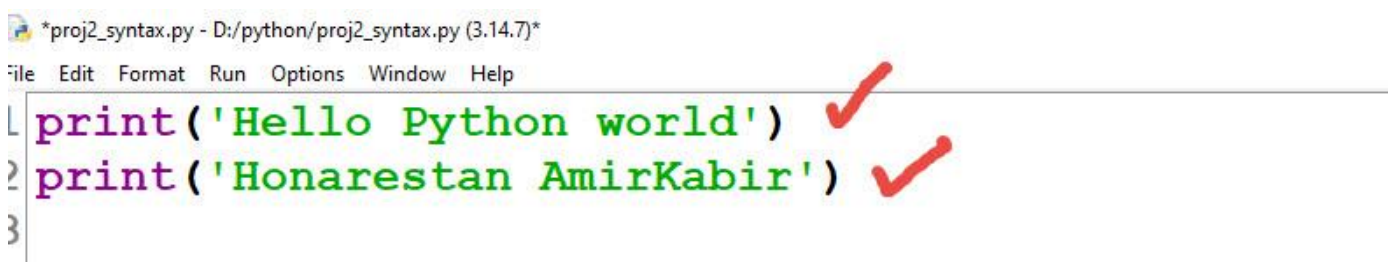
>>>
===== RESTART: D:/python/proj2_syntax.py =====
Hello Python world
Honarestan AmirKabir

```

(شکل ۲)

نکته: همانطور که در شکل فوق مشاهده می کنید یکی از مواردی که باید در ساختار برنامه نویسی به زبان پایتون رعایت شود این است که هرگاه بخواهیم رشته ای را به عنوان پیغام در خروجی نمایش دهیم باید آن رشته را داخل علامت دبل کوتیشن " " قرار دهیم، حال سوال این است که آیا می توانیم به جای علامت دبل کوتیشن از علامت تک کوتیشن استفاده کنیم در پاسخ باید بگوییم که بله این امکان وجود دارد اما در ورژن های جدید پایتون بیشتر از علامت کوتیشن برای نمایش رشته یا پیغام استفاده می شود.

در شکل های زیر حالت های مختلفی که می توانیم از کوتیشن و یا تک کوتیشن استفاده کنیم را نمایش می دهیم.



```

*proj2_syntax.py - D:/python/proj2_syntax.py (3.14.7)*
File Edit Format Run Options Window Help

1 print('Hello Python world') ✓
2 print('Honarestan AmirKabir') ✓
3

```

اگر در طرفین تک کتیشن یا کتیشن باشد حالت های صحیح و خطا را در شکل های زیر مشاهده می کنیم.

The screenshot shows the IDLE Shell window with the following code and output:

```
print("Hello "Python" world")
print("Honarestan ' AmirKabir")
```

The output displayed is:

```
Hello "Python" world
Honarestan ' AmirKabir
```

The screenshot shows the IDLE Shell window with the following code:

```
1 print('Hello 'Python' world')
2 print("Honarestan " AmirKabir")
3
4
```

A red 'X' marks the closing quote on line 2. A syntax error dialog box is shown with the message: "SyntaxError: unterminated string literal (detected at line 2)".

نکته: اگر ما در ابتدا و انتهای پرانتز جلوی دستور Print از سه تا کتیشن استفاده کنیم (به استفاده از سه کتیشن اصطلاحاً تریپل کتیشن گفته می شود) می توانیم مابین علامت تریپل کتیشن ابتدا و انتهای پرانتز از هر علامت دیگری نیز استفاده کنیم.

The screenshot shows the IDLE Shell window with the following code:

```
print("""Hello Python world's """)
```

The output displayed is:

```
=== RESTART: D:/python/proj2_syntax.py ===
Hello Python world's
```

همچنین به کمک Triple Cotation ما می توانیم رشته ها یا پیغام ها را به صورت پاراگرافی نیز چاپ کنیم.

```
print("""Hello
      Python
      world's """)
```

The screenshot shows two windows. The left window is a Python script named `proj2_syntax.py` with the following code:

```
print("""Hello
      "Python"
      world's """)
```

Red arrows point from the string literals in the code to the output in the right window. The right window is the IDLE Shell, showing the output of the script:

```
=== RESTART: D:/python/proj2_syntax.py ===
Hello
      "Python"
      world's
```

نکته: اگر در داخل پرانتز دستور `print` فقط یک عدد بنویسیم مثلاً اگر عدد ۱۵ را بنویسیم و سپس آن را اجرا کنیم برنامه در خروجی عدد ۱۵ را برای ما چاپ می کند و از این عدد می توانیم بعد ها در محاسبات ریاضی نیز استفاده نماییم، اما اگر عدد ۱۵ را در داخل دبل کتیشن قرار دهیم و سپس برنامه را اجرا کنیم در خروجی برنامه عدد ۱۵ برای ما چاپ نمی شود و صرفاً پیغام ۱۵ را به عنوان یک مقدار رشته ای برای ما چاپ می کند و ما نمی توانیم از این رشته ی عددی در محاسبات استفاده کنیم.

The screenshot shows the same script as before, but with the following code:

```
print(15)
print(15+5)
```

Red arrows point from the numeric expressions to the output in the IDLE Shell:

```
=== RESTART: D:/python/proj2_syntax.py ===
15
20
```

The screenshot shows the script with the following code:

```
print("15")
print("15+5")
```

Red arrows point from the string literals to the output in the IDLE Shell:

```
15
15+5
```

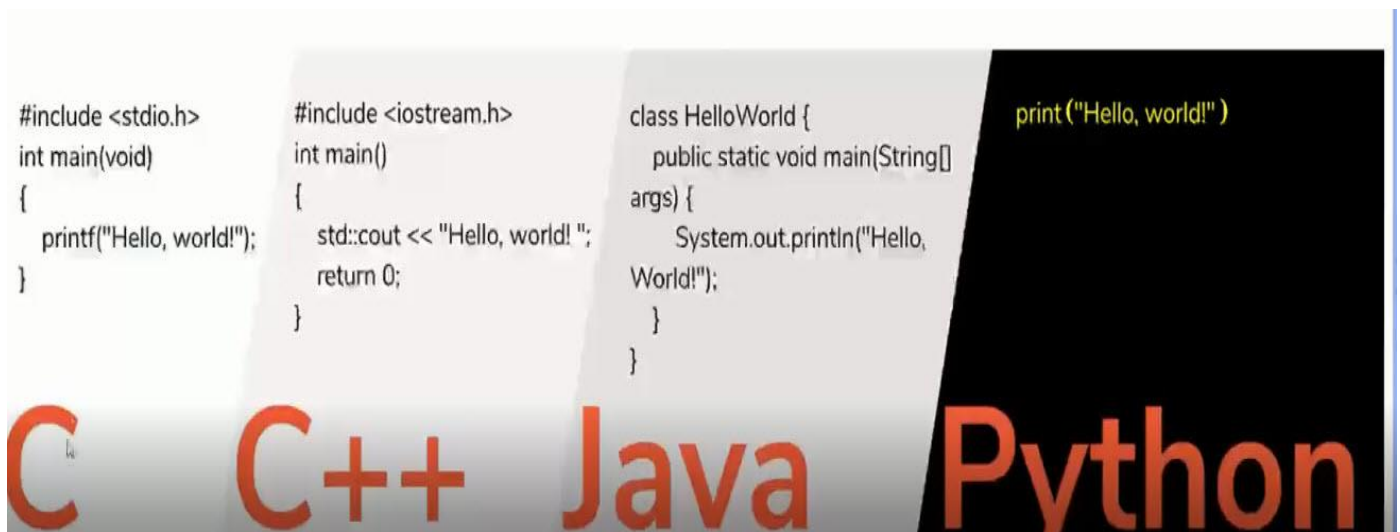
نکته: دستور `print` این امکان را به ما می دهد که هم پیغام و هم عدد (اعداد صحیح و اعشاری) را با هم چاپ نماییم، برای این منظور به صورت زیر عمل می کنیم که ابتدا کلمه کلیدی `print()` سپس پیغام مورد نظر خود را در داخل کتیشن نوشته، سپس از یک **ویرگول** و **Space** استفاده نموده و بعد عدد مورد نظرمان را می نویسم. (در زبان پایتون برای نمایش عدد اعشاری از نقطه به عنوان اعشار استفاده می کنیم).



```
proj2_syntax.py - D:/python/proj2_syntax.py (3.14.7)
File Edit Format Run Options Window Help
1 print("Pi=", 3.14)
2
3
4

IDLE Shell 3.14.7
File Edit Shell Debug Options Window Help
=== RESTART: D:/python/proj2_syntax.py ===
Pi= 3.14
>>>
Ln: 50 Col: 0
```

نکته: همانطور که در ابتدا گفتیم ساختار دستور یا دستورات در زبان پایتون بسیار ساده می باشد و مشابه زبان انگلیسی می باشد در شکل زیر ساختار دستور یا دستورات را در سایر زبان های برنامه نویسی مشاهده می کنید و می بینید که همین برنامه Hello World را اگر بخواهیم با زبانهای برنامه نویسی دیگر نظیر زبان C، C++، و یا Java بنویسیم چه ساختاری را باید رعایت کنیم (مثلا در آن زبان ها از علامت آکولاد باز و بسته باید استفاده کنیم و یا اینکه از علامت سمی کالن باید استفاده کنیم و ...) تا بتوانیم پیغام Hello World را در خروجی آن برنامه ها ببینیم.



<pre>#include <stdio.h> int main(void) { printf("Hello, world!"); }</pre>	<pre>#include <iostream.h> int main() { std::cout << "Hello, world! "; return 0; }</pre>	<pre>class HelloWorld { public static void main(String[] args) { System.out.println("Hello, World!"); } }</pre>	<pre>print("Hello, world!")</pre>
C	C++	Java	Python

تورفتگی (indent):

ما برخی از دستوراتِ مان در زبان پایتون یک مجموعه دستوراتی را در درون خودشان دارند که اصطلاحاً گفته می شود هر دستوری یک فرمت استفاده یا شکل استفاده دارد، به عنوان مثال دستور `if` که هنوز آن را درس نداده ایم دارای یک فرمت استفاده به صورت شکل زیر دارد:

شرط: if

بلاک دستورات

همانطور که در فرمت دستور `if` مشاهده می کنید ما موقع استفاده از این دستور ابتدا باید از کلمه کلیدی `if` استفاده کنیم سپس جلوی آن باید شرط خود را نوشته و سپس جلوی عبارت شرط خود باید علامت دو نقطه قرار دهیم و سپس در زیر علامت دو نقطه می توانیم از یک دستور و یا چندین دستور استفاده نماییم که به مجموعه دستوراتی که در خطوط جداگانه در زیر دو نقطه مورد استفاده قرار می دهیم اصطلاحاً یک بلاک دستورات می گوئیم که مربوط به آن دستور `if` می باشند حال نکته بسیار مهم و اساسی این است که اگر توجه کنید همه ی دستورات مربوط به این بلاک، همگی به اندازه ۴ تا `space` جلوتر از کلمه کلیدی `if` نوشته شده اند که اصطلاحاً به این تورفتگی `indent` می گوئیم. در شکل زیر تورفتگی را در دستور `if` مشاهده می کنیم. بنابراین همانطور که در شکل زیر مشاهده می کنید در زبان پایتون این تورفتگی ها و نظم آن ها بسیار مهم می باشد و حتماً باید این موضوع را رعایت کنیم.

بلاک دستورات if

if x > 0:

Indent
تورفتگی

```
print("hello")
print("Positive")
print("salam")
```

نکته: در واقع به جای آکولاد باز و بسته هایی که ما در زبان های دیگر نظیر زبان C، C++ و ... داریم ما باید حتما این تورفتگی ها را در زبان پایتون با دقت رعایت کنیم. همانطور که گفتیم میزان تورفتگی ها معمولا به اندازه ۴ تا space می باشد البته می توان میزان تورفتگی را به اندازه ۲ تا space یا ۱ space و یا ۵ تا space هم تنظیم کرد اما استاندارد میزان تورفتگی ها در زبان پایتون مضربی از عدد ۴ می باشد. یعنی اگر مجبور باشیم در همان برنامه تورفتگی دیگری را مورد استفاده قرار دهیم تورفتگی سری دوم در آن برنامه باید به اندازه ۸ تا space نسبت به بلاک دستور باشد و به همین ترتیب.

بلاک دستورات if

if x > 0:

Indent
تورفتگی

```
print("hello")
print("Positive")
print("salam")
```

Errorr

متغیرها (Variables) و انواع داده (Data Type)

در جدول زیر معروفترین انواع داده ها را در زبان برنامه نویسی پایتون مشاهده می نمایید.

Python Data Types معروفترین انواع داده در پایتون				
Text Type:	str	"iran" 'x'	رشته ای	نوع متنی:
Numeric Types:	int	25, -5, 0	صحیح	انواع عددی:
	float	3.14, 0.23	اعشاری	

همانطور که در جدول فوق مشاهده می کنید می توان گفت که هر داده ای در یکسری از طبقه بندی ها قرار می گیرد، به طور کلی دو نوع از انواع معروفترین داده ها عبارتند از: ۱- **داده های رشته ای** که این داده ها را با کلمه کلیدی **str** تعریف می کنیم. ۲- **داده های عددی**، خود داده های عددی در دو دسته ی داده های **عدد صحیح** که با کلمه کلیدی **int** تعریف می شوند و **داده های اعشاری** که با کلمه کلیدی **float** تعریف می شوند. البته انواع دیگری داده ها شامل **list**، **set**، **tuple**، **dict** و **bool** را نیز داریم که بعداً هر یک از داده ها را معرفی خواهیم کرد.

نکته: با توجه به این توضیحات اگر به عنوان مثال در پنجره ی کد نویسی فایل پایتونی خود، ما عبارتی به صورت **name="dana"** را بنویسیم در واقع قرار است که ما کلمه ی **dana** را در متغیری با نام **name** ذخیره کنیم (یعنی محتوای متغیر با نام **name** در حال حاضر با مقدار **dana** پر شده است)، در این حالت کامپیوتر می آید و یک قسمتی از حافظه ی خود را برای این مقدار در نظر می گیرد. حال اگر به عنوان مثال در خط بعدی عبارت **salary=500** را نیز بنویسیم مجدداً کامپیوتر می آید و یک قسمتی از حافظه ی خود را برای این مقدار نیز در نظر می گیرد. (شکل زیر)



برای اعداد اعشاری نیز همان توضیحاتی که بیان کردیم صدق می کند، یعنی اگر به عنوان مثال در خط بعدی عبارت $Pi=3.14$ را بنویسیم مجدداً کامپیوتر می آید و یک قسمتی از حافظه ی خود را برای این مقدار نیز در نظر می گیرد و محتوای متغیر را برابر با ۳,۱۴ در نظر می گیرد. (شکل زیر)



نکته مهم: سوال مهم این است که ما چرا این مقادیر را در داخل متغیرهایی با نام های `salary`، `name` و یا `pi` می ریزیم؟ در پاسخ باید بگوییم یکی از **بهترین مزیت های استفاده از متغیر** این است که ما می توانیم محتوایی که داخل متغیر قرار داده ایم را براحتی تغییر دهیم و مقادیر جدیدی را به عنوان محتوای داخل متغیر در نظر بگیریم. یعنی می توانیم به جای کلمه `dana` کلمه `ali` را داخل متغیر `name` قرار دهیم بدین ترتیب آخرین مقداری که در حال حاضر داخل متغیر `name` قرار می گیرد کلمه `ali` خواهد بود و مقدار قبلی پاک

می شود، به همین علت است که در محیط کد نویسی به جای اینکه ما بیاایم از کلماتی مانند **dana** یا **ali** و ... استفاده کنیم می آایم و از نام متغیر ها استفاده می کنیم، بنابراین می توانیم متغیر را اینگونه تعریف کنیم که:

تعریف متغیر: مکانی موقت در حافظه کامپیوتر می باشد که می توانیم مقادیر دلخواهی را در داخل آن قرار دهیم.

```

prog3_variable.py - D:/python/prog3_varia
IDLE Shell 3.14.7
File Edit Shell Debug
1 name="dana"
2 salary=500
3 pi=3.14
4 print(name)
5 print(salary)
6 print(pi)
7
>>>
===== RESTART: D:/python/prog3_variab
e.py =====
dana
500
3.14
>>>
Ln: 8 Col: 0

```

قواعد نام گذاری متغیرها در پایتون

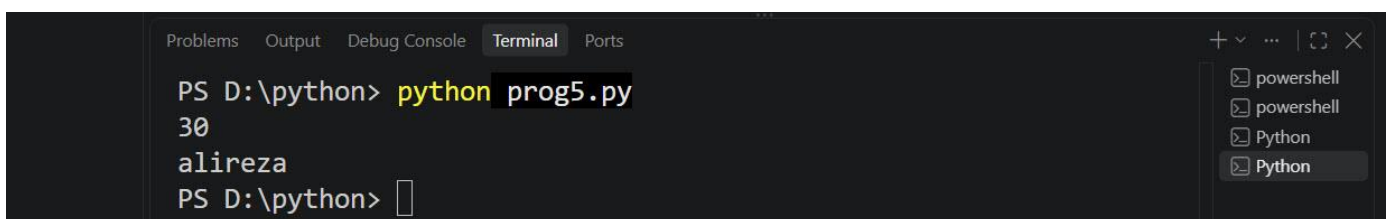
- نام متغیر نمی تواند شامل کاراکتر فاصله باشد. به طور مثال نمی توان متغیری با نام **is valid** تعریف کرد.
- نام گذاری متغیرها در پایتون باید با یک حرف یا زیر خط آغاز شود. مثل (**_num** , **person1**)
- نام متغیر می تواند شامل حروف، اعداد و زیر خط باشد.
- نام متغیرها به حروف کوچک و بزرگ حساس است **age**، **Age** و **AGE** سه متغیر متفاوت هستند.
- نام متغیر نباید هیچ یک از کلمات کلیدی پایتون باشد. نام های **import, if, const, var, for** به عنوان نام متغیر اشتباه است. چون این کلمات از کلمات کلیدی پایتون هستند.
- نکته: کلمه **کلیدی (Keyword)** در پایتون به واژه هایی گفته می شود که خود زبان پایتون برای دستورها و ساختارهایش رزرو کرده است. این کلمات معنای از پیش تعریف شده دارند و به همین دلیل نمی توان از آن ها به عنوان نام متغیر، تابع یا کلاس استفاده کرد.

مفهوم عملگر یا (operator) و عملوند یا (operand)

عملگر یا (operator) نمادهای ویژه ای هستند که عملیات خاصی را روی مقادیر یا متغیرها انجام می دهند. مثلا $B + A$ یا $D * C$ یا $۸۲ / ۴$ که به $+$ ، $*$ و $/$ (operator) گفته می شود. عملوند یا (operand) مقدار یا مقدارهایی هستند که عملگرها روی آن کار می کنند. مثلا در عبارت $B + A$ ، علامت $+$ عملگر، و A و B عملوند هستند. عملوندهای عملگر $+$ در پایتون می توانند هر دو عددی یا هر دو رشته ای باشند که در این صورت در حالت اول جمع ریاضی و در حالت دوم الحاق یا (concatination) اتفاق می افتد.

مثال: (برنامه زیر در نرم افزار **vscode** نوشته و اجرا شده است، در جلسه بعد این نرم افزار را نصب نموده و نحوه نوشتن برنامه و اجرای برنامه های پایتون را در این نرم افزار مورد بررسی قرار خواهیم داد).

```
a = 10
b = 20
c = a + b
print(c)
a = "ali"
b = "reza"
c = a + b
print(c)
```



```
PS D:\python> python prog5.py
30
alireza
PS D:\python>
```

برخی از انواع عملگرها در پایتون:

جدول ۱ برخی از عملگرهای ریاضی را نشان می دهد که می توانید از آنها در انجام محاسبات ریاضی استفاده کنید.

جدول ۱

عملگر ریاضی	کاربرد
+	جمع
-	تفریق
*	ضرب
/	تقسیم
%	باقی مانده تقسیم

شکل ۲۲

نکته: انواع دیگری از عملگرها نیز وجود دارند که در مباحث آینده به معرفی آن ها خواهیم پرداخت.

نمایش خروجی با وضوح بهتر: همیشه سعی کنید زمانی که می خواهید با دستور `print` محتوای متغیری را چاپ کنید ابتدا مطابق شکل زیر پیغامی را داخل کتیشن قرار دهید و سپس بعد از `space` و ویرگول نام متغیر را بنویسید بدین ترتیب کاربری که برنامه شما را می بیند متوجه می شود که آن متغیر چه چیزی را به ما نشان می دهد.

```

1 name="dana"
2 salary=500
3 pi=3.14
4 print("name=" ,name)
5 print("salary" ,salary)
6 print("pi=" ,pi)
7

```

```

===== RESTART: D:/python/prog3_variable.py =====
name= dana
salary 500
pi= 3.14

```

همانطور که گفتیم متغیر مکانی موقت در حافظه کامپیوتر می باشد که آخرین مقدار خود را در خروجی به ما نشان می دهد اگر در برنامه بالا این بار در داخل متغیر `name` کلمه `ali` را قرار دهیم و برنامه را اجرا

کنیم مشاهده می کنیم که آخرین مقداری که به عنوان محتوای متغیر `name` در نظر می گیرد کلمه `ali` می باشد. (شکل زیر)

```

prog3_variable.py - D:/python/prog3_variable.py (3.14.7)
File Edit Format Run Options Window Help

1 name="dana"
2 salary=500
3 pi=3.14
4 print("name=" ,name)
5 print("salary" ,salary)
6 print("pi=" ,pi)
7 name="ali"
8 print("name=" ,name)
9

===== RESTART: D:/python/prog3_variable.py =====
name= dana
salary 500
pi= 3.14
name= ali

```

نکته: اگر بخواهیم بدانیم که هریک از داده هایی که در متغیرهای `name`، `salary` و یا `pi` قرار داده ایم از چه نوعی می باشند یا به عبارتی `data Type` آنها از چه نوعی می باشد می توانیم از دستور `print` به صورت زیر استفاده نماییم. البته در جلسات بعدی با انواع دیگری از `data Type` نیز آشنا خواهیم شد.

Print(type(نام متغیر))

```

name="dana"
salary=500
pi=3.14
print("name=" ,name)
print("salary" ,salary)
print("pi=" ,pi)
name="ali"
print("name=" ,name)
print(type(name))
print(type(salary))
print(type(pi))

```

خروجی:

The screenshot shows the Python IDLE environment. On the left, the editor displays a file named `prog3_variable.py` with the following code:

```
1 name="dana"
2 salary=500
3 pi=3.14
4 print("name=" ,name)
5 print("salary" ,salary)
6 print("pi=" ,pi)
7 name="ali"
8 print("name=" ,name)
9 print(type(name))
10 print(type(salary))
11 print(type(pi))
12
```

On the right, the IDLE Shell window shows the output after running the script:

```
==== RESTART: D:/python/prog3_variable.py ===
name= dana
salary 500
pi= 3.14
name= ali
<class 'str'>
<class 'int'>
<class 'float'>
>>>
```

نکته: اگر ما بخواهیم موقع اجرای برنامه، خط یا خطوطی از برنامه ما اجرا نشوند می توانیم به کمک علامت **#** که در ابتدای آن خط قرار می دهیم اصطلاحاً آن خط را **comment** کنیم یعنی موقع اجرای برنامه آن خط برای ما اجرا نمی شود، با انجام این کار هیچ مشکلی برای برنامه ی ما به وجود نمی آید ، و هر زمانی هم که بخواهیم مجدداً آن خط موقع اجرای برنامه برای ما اجرا شود می توانیم آن را از حالت **comment** خارج کنیم یعنی علامت **#** را از ابتدای آن خط یا خطوط حذف می کنیم.

The screenshot shows the Python IDLE environment. On the left, the editor displays the same file `prog3_variable.py` but with some lines commented out:

```
1 name="dana"
2 salary=500
3 pi=3.14
4 print("name=" ,name)
5 print("salary" ,salary)
6 print("pi=" ,pi)
7 #name="ali"
8 #print("name=" ,name)
9 #print(type(name))
10 print(type(salary))
```

On the right, the IDLE Shell window shows the output after running the script:

```
==== RESTART: D:/python/prog3_varia
ble.py ====
name= dana
salary 500
pi= 3.14
<class 'int'>
```

A red callout box labeled "comment" points to the commented-out lines in the script.